MC

mc pre

也就是说，它不是在学“怎么行动”，而是在评估：给定一个固定策略 π，估计每个状态 s 的价值

用一个完整例子走一遍

假设一个 episode 是：
S0=A, R1=0
S1=B, R2=0
S2=A, R3=1
S3=Terminal

也就是：A → B → A → Terminal

奖励分别是：
R1 = 0
R2 = 0
R3 = 1

设：
γ=0.9

从后往前算。

第一步：t = 2	此时状态是：S2=A	更新回报：G=0.9×0+R3=1	但是 A 在前面 S0 已经出现过了因为是 first-visit MC，所以跳过这次 A。
第二步：t = 1	此时状态是：S1=B	更新回报：G=0.9×1+R2=0.9	B 之前没有出现过，所以更新：V(B)←V(B)+α[0.9−V(B)]
第三步：t = 0	此时状态是：S0=A	更新回报：G=0.9×0.9+R1=0.81	这是 A 第一次出现，所以更新：V(A)←V(A)+α[0.81−V(A)]

A 的目标回报是 0.81
B 的目标回报是 0.9

所以这个 episode 最终用来更新的是：

```
Plain
复制
initialize V(s) arbitrarily

repeat for each episode:
    generate episode using policy pi:
        S0, A0, R1, S1, A1, R2, ..., ST

    G = 0
    for t = T-1 down to 0://从后往前算--因为经验数据长这样S0, A0, R1, S1, A1, R2, S2, A2, R3, ..., ST
        G = gamma * G + R_{t+1}//所以对于时间 t，回报是从 Rt+1开始算的

        if first-visit MC://first出现优先（这个不用关心，并列的有every）
            if S_t appeared earlier in this episode:
                continue

        V(S_t) = V(S_t) + alpha * [G - V(S_t)]//最好的理解就是放在代码里 直观的看出这是赋值：等价于V(St)←(1-α)V(St)+αG
                不要完全相信这一次 episode 的回报 G，只把 V(St) 往 G 的方向拉近一点。
```

MC con

```
Plain
复制
initialize Q(s,a) arbitrarily
initialize pi as epsilon-greedy(Q)

repeat for each episode:
    generate episode using current policy pi:
        S0, A0, R1, S1, A1, R2, ..., ST
    G = 0
    for t = T-1 down to 0:
        G = gamma * G + R_{t+1}
        if first-visit MC:
            if (S_t, A_t) appeared earlier in this episode:
                continue

        Q(S_t,A_t) = Q(S_t,A_t) + alpha * [G - Q(S_t,A_t)]

        pi(.|S_t) = epsilon-greedy distribution based on Q(S_t,.)
```

公式是：

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[G - Q(S_t, A_t)]$$

它的意思是：

> 我以前估计在 S_t 下做 A_t 的经值是 $Q(S_t, A_t)$，现在这个 episode 真实跑出来的回报是 G，所以我把 $Q(S_t, A_t)$ 往 G 拉近一点。

例如：

```
Q(A, right) = 2
G = 10
alpha = 0.1
```

更新后：

$$Q(A, right) = 2 + 0.1(10 - 2) = 2.8$$

↓

也就是说：

> 这次发现从 A 选择 right 之后结果不错，所以提高 $Q(A, right)$。

为什么还要更新 policy？
pi(.|S_t) = epsilon-greedy distribution based on Q(S_t,.)
意思是：根据当前状态 St下所有动作的 Q 值，重新调整策略。

采用 ε-greedy 策略

假设：epsilon = 0.1 动作数量 = 3

那么策略大概是：
动作 选择概率
right 0.9+0.1/3
left 0.1/3
up 0.1/3

也就是说：90% 的概率利用当前最优动作，10% 的概率探索其他动作。

当前 Q 表
↓
根据 Q 生成 epsilon-greedy 策略 pi
↓
用 pi 跑一个完整 episode
↓
算每个 (S_t, A_t) 的完整回报 G_t
↓
用 G_t 更新 Q(S_t, A_t)
↓
根据新的 Q 改进 pi
↓
继续下一轮

为什么第一种不能直接用V来更新动作 这是值迭代和策略迭代的区别吗

在 model-free 的 MC control 里，只有 V(s) 不足以直接判断哪个动作更好。

如果你只知道：
V(s)=5
这个数字只表示：
按照当前策略 π，在状态 s 的**平均价值**是 5。
所以只靠 V(s)，你不知道该让策略更偏向 left 还是 right。

V(s) 本质上已经把动作“平均掉了”

状态价值函数是：
$V\pi(s) = E\pi[Gt \mid St = s]$

这里的下标 π 很关键。它的意思是：
从状态 s 开始，动作是按照策略 π 选的。

所以 MC control 为什么更新 $Q(s,a)$？

MC control 要做策略改进：
$\pi(s) \leftarrow argamax Q(s,a)$要选 Q 最大的动作。

那什么时候可以用 V(s) 来改进策略？

可以，但前提是你知道环境模型。也就是说，你得知道：p(s',r | s,a)

也就是：在状态 s 下选动作 a，会以多大概率到达下一个状态 s'，并获得奖励 r。

如果你知道这个模型，就可以用 V 做一步前瞻：π(s)=argamaxs',∑p(s',r | s,a)[r+γV(s')]

意思是：对每个动作 a，看看它可能带你到哪些下一个状态 s'，然后用 r+γV(s') 评价这个动作。

这时候确实可以用 V 来改进策略。

但是 MC 方法通常是 model-free，它不知道环境转移概率 p(s',r | s,a)。它只能通过实际采样 episode 来估计回报。

所以在 model-free control 里，更自然的做法是直接学：
Q(s,a)

因为 Q(s,a) 已经把“选动作 a 之后会发生什么”这件事包含进去了。